

APPLIED AI · 5350 · FALL 2026

Vibe Coding with Python

Using natural language to create, explain, debug, and improve software —
with GitHub Copilot and a local IBM Granite model.



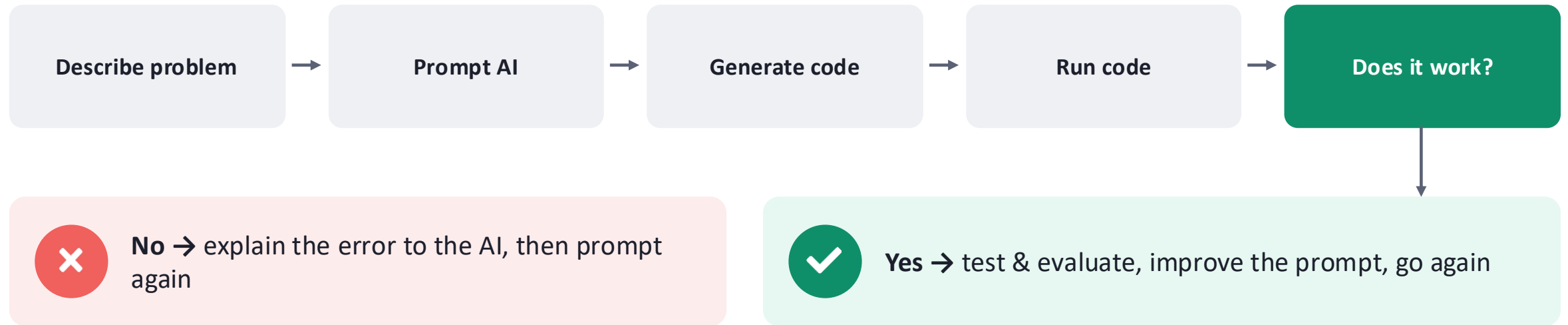
Ivan Portilla



UNIVERSITY OF COLORADO BOULDER

What is vibe coding?

Describe what you want in natural language; the AI writes much of the implementation.



The human is still responsible for deciding:

What should the program do — and is the result correct?



The goal is not just to have AI generate code

In this lab you will learn to:

1

Describe

Clearly describe a problem

2

Generate

Ask an AI to generate a solution

3

Understand

Understand the generated code

4

Test

Test whether the solution works

5

Critique

Identify mistakes or weaknesses

6

Improve

Ask the AI to improve the solution

7

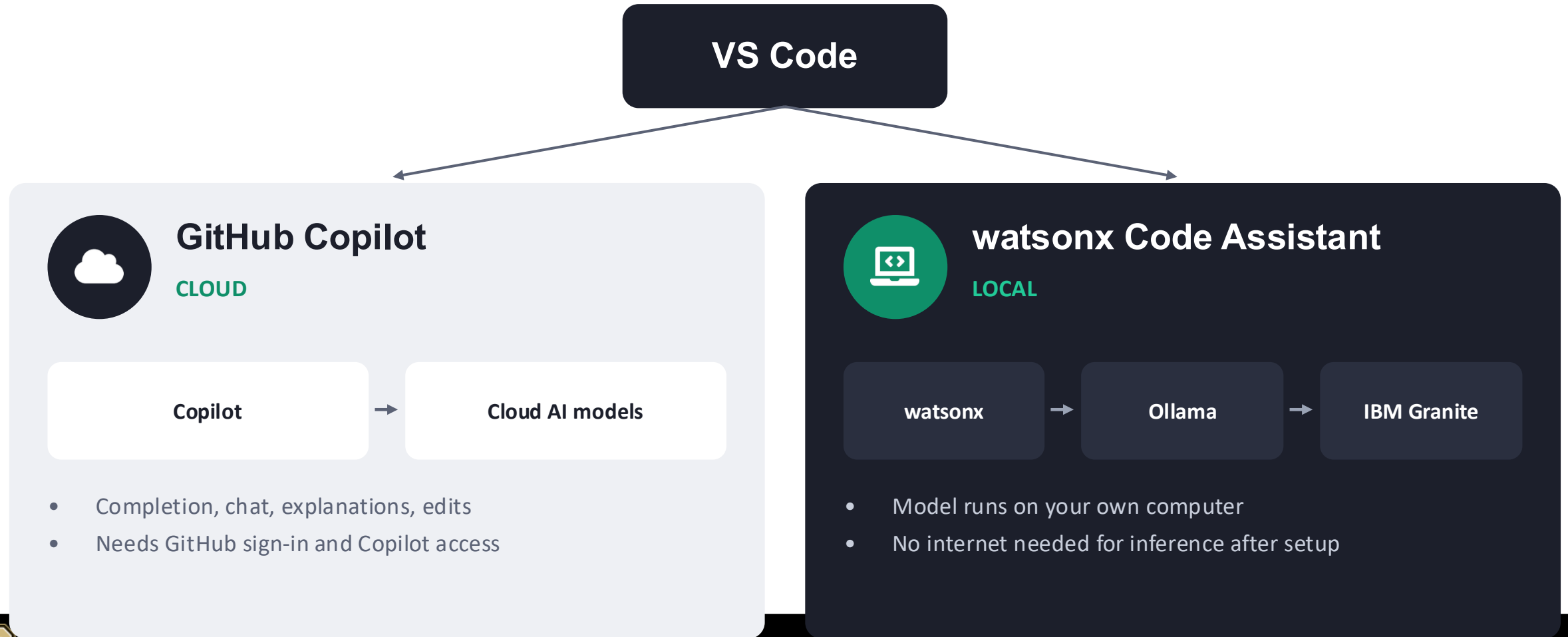
Compare

Compare different algorithms and approaches



Two AI coding assistants, one editor

Both run as extensions inside Visual Studio Code.



Setup 1 · GitHub Copilot

Install, sign in, and test in a few minutes.

1

Install VS Code

code.visualstudio.com

2

Install the extension

Extensions (Ctrl/Cmd + Shift + X) → search “GitHub Copilot” → Install (publisher: GitHub)

3

Sign in to GitHub

Authorize VS Code in the browser. Copilot Free and verified student plans are available.

4

Test it

Create test.py, type a comment and a function name, accept suggestions with Tab — or open Copilot Chat

TEST.PY · PRESS TAB TO ACCEPT

```
# Create a function that generates  
# the first n Fibonacci numbers  
  
def fibonacci█
```



Copilot Chat: “Write a Python function that generates the first 20 Fibonacci numbers.”

Chat can also explain code, propose edits, and help validate files.



Setup 2 · Ollama + IBM Granite

Ollama is the local runtime that serves the model on your machine.

TERMINAL

```
# Install (or download from ollama.com)
brew install ollama          # macOS
# Linux:
curl -fsSL https://ollama.com/install.sh | sh

# Verify
ollama --version
ollama list
ollama serve                # if not already running

# Download + chat with Granite (several GB)
ollama run granite-code:8b
>>> Write a Python function that
    calculates the Fibonacci sequence.
>>> /bye
```

Listens on

127.0.0.1:11434

Model for this lab

granite-code:8b



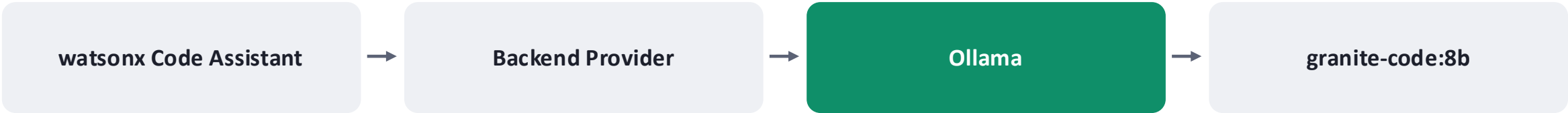
“address already in use”

usually means Ollama is already running — just continue.



Setup 3 · Point watsonx Code Assistant at Ollama

Install the IBM extension, then change two settings.



VS Code setting	Value	Note
Wca: Backend Provider	<code>ollama</code>	Required — not the IBM Cloud backend
Wca › Local: Code Gen Model	<code>granite-code:8b</code>	Default local model
Local API host	<code>127.0.0.1:11434</code>	Change only if Ollama runs elsewhere

Can't connect

```
ollama serve, then ollama run granite-code:8b
```

Model not found

```
ollama pull granite-code:8b
```

Is it loaded?

```
ollama list · ollama ps
```



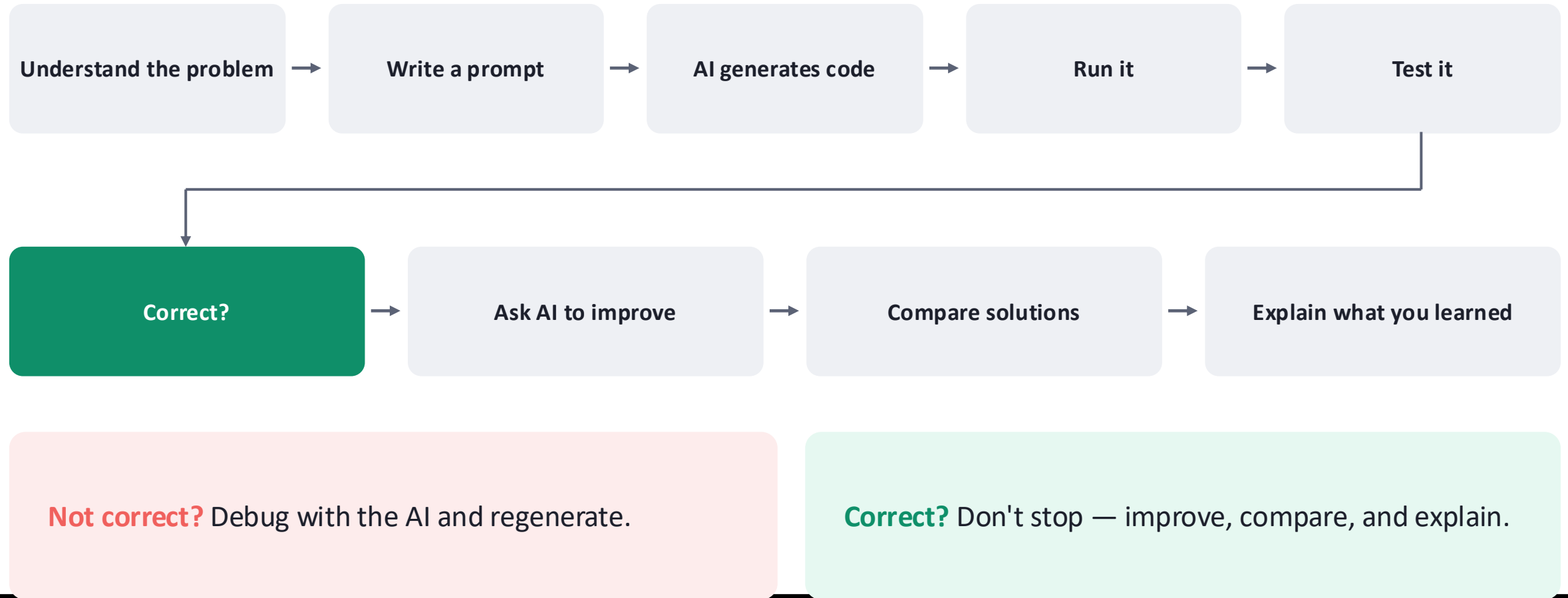
Cloud vs. local at a glance

Both are good for this lab — the difference is where the model runs.

	GitHub Copilot	IBM + Ollama
IDE	VS Code	VS Code
AI interface	GitHub Copilot	watsonx Code Assistant
Model location	Cloud	Your computer
Local runtime	—	Ollama
Model	Copilot-supported models	IBM Granite
Internet dependency	Generally yes	Not required for inference after setup

The vibe coding process

Follow the same loop for every exercise.



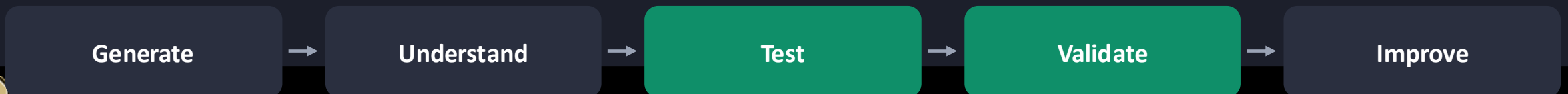
Don't just accept the code

AI-generated code can ...

- ⚠ Contain bugs
- ⚠ Use inefficient algorithms
- ⚠ Misunderstand requirements
- ⚠ Produce incorrect results
- ⚠ Introduce security problems
- ⚠ Use libraries incorrectly
- ⚠ Look right but be wrong

**Running without errors
does not mean the
program is correct.**

A better workflow



From code generator to coding collaborator

Instead of only “Write the code”, try:

“Explain this code to me.”

“Is there a more efficient algorithm?”

“What is the time complexity?”

“What edge cases am I missing?”

“Generate test cases for this function.”

“Try to break this program.”

“Compare this with another approach.”

“Explain the tradeoffs between the two.”

“How would you make this production quality?”



Same prompts, two assistants

Run the same sequence in Copilot and in watsonx + Granite, then compare.

1

"Write a Python program that generates the Fibonacci sequence."

2

"Explain your algorithm."

3

"Can you make it more efficient?"

4

"Generate test cases that could break this implementation."

CONSIDER

- Did they generate the same algorithm?
- Which explanation was clearer?
- Did either make mistakes?
- Different Python styles?
- Which generated better tests?
- How did local compare with cloud?



Five exercises, one pattern

Run the original → ask the AI to explain → improve → test → compare.

1



Fibonacci

Recursive vs. iterative

2



Duplicates

Nested loops vs. set

3



Search

Linear vs. binary

4



Common word

Counting & visualizing

5



Passwords

Strength checker

Starter code for every exercise is in `vibe_coding_applied_ai.ipynb`



1 · Fibonacci: recursive vs. iterative

0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

RECURSIVE — SIMPLE

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

ITERATIVE — KEEPS LAST TWO VALUES

```
def fibonacci_fast(n):  
    sequence = []  
    a, b = 0, 1  
    for _ in range(n):  
        sequence.append(a)  
        a, b = b, a + b  
    return sequence
```



“Explain why the iterative version is faster than the recursive one.”



“Modify the program to measure the execution time of both.”

THINK ABOUT

Why does recursion recalculate the same values again and again?



2 · Finding duplicates: nested loops vs. set

`numbers = [4, 7, 2, 4, 9, 7, 10] → [4, 7]`

NESTED LOOPS — COMPARE EVERY PAIR

```
for i in range(len(numbers)):
    for j in range(i + 1, len(numbers)):
        if numbers[i] == numbers[j]:
            if numbers[i] not in duplicates:
                duplicates.append(numbers[i])
```

SET — ONE PASS, FAST LOOKUPS

```
seen, duplicates = set(), set()
for number in numbers:
    if number in seen:
        duplicates.add(number)
    seen.add(number)
```

Next number →

Seen before?

Yes → duplicates

No → add to seen



“Why is the set version faster?”



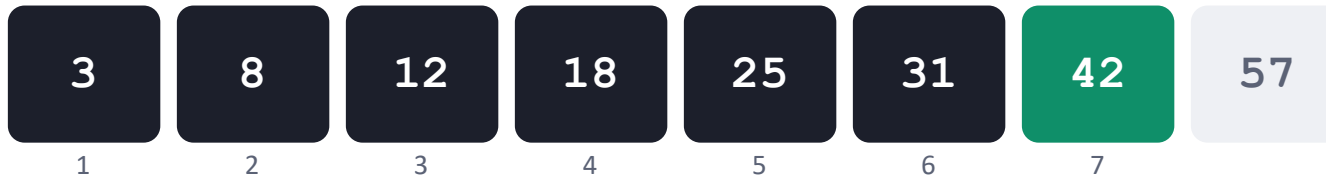
“Compare both on 100,000 random numbers.”



3 · Searching: linear vs. binary

Target 42 in the sorted list [3, 8, 12, 18, 25, 31, 42, 57]

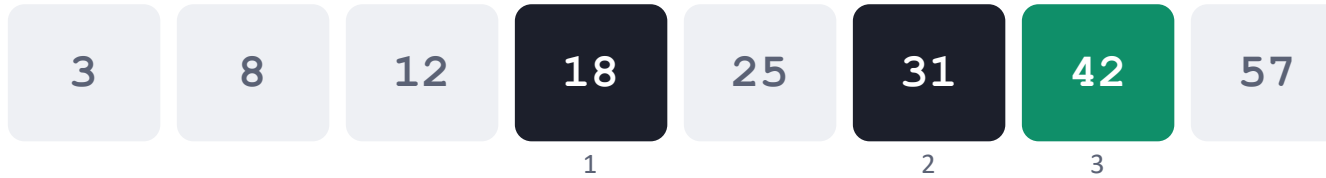
Linear — check one at a time



7

values checked
by linear search

Binary — halve the range each step



3

values checked
by binary search



"Modify binary search so it prints every value it checks."

Binary search only works on a sorted list.



4 · Most common word: grow it by prompting

"AI is changing software development. AI can generate code. AI can also make mistakes."



1

"Make punctuation not affect the count."

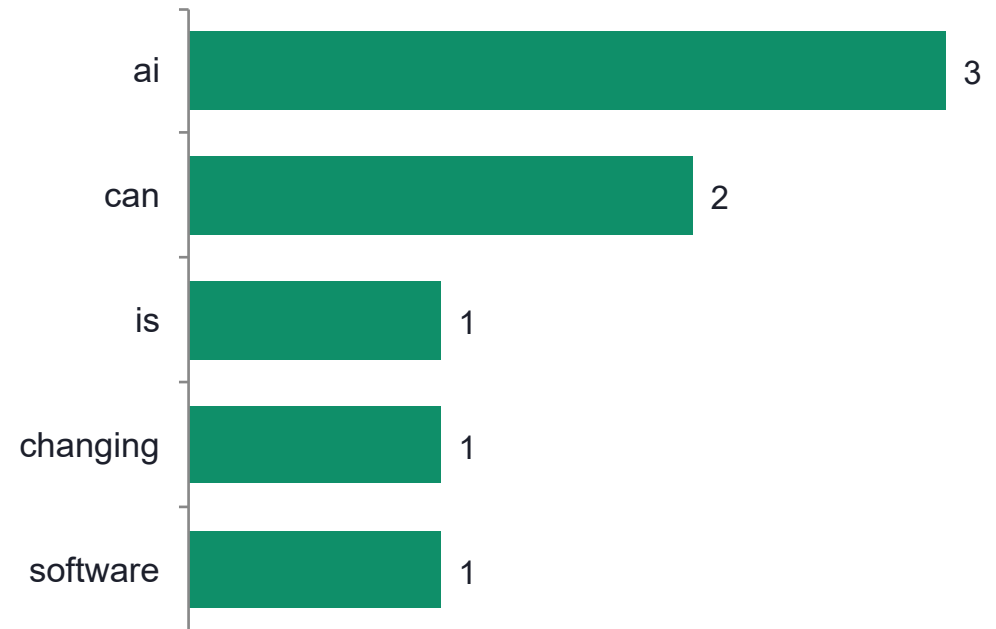
2

"Show the five most common words and their counts."

3

"Create a bar chart of the top five words."

Top five words (after cleaning)



5 · Cybersecurity: password strength checker

Check basic password characteristics — then make the feedback useful.

RULES



Minimum length (12+)

starter



Uppercase letter

starter



Lowercase letter

starter



Number

starter



Special character

you add



Not a common password

you add



“Require at least one special character.”



“Reject passwords from a list of commonly used passwords.”



“Instead of only weak or strong, explain which requirements failed.”



Never enter a real password.

Use a made-up one like VibeCoding2026 — especially with a cloud assistant.



Final reflection

Choose one exercise and answer:

1 What was your original prompt?

2 What code did the AI generate?

3 Did it work correctly the first time?

4 How did you test it?

5 What did you ask the AI to improve?

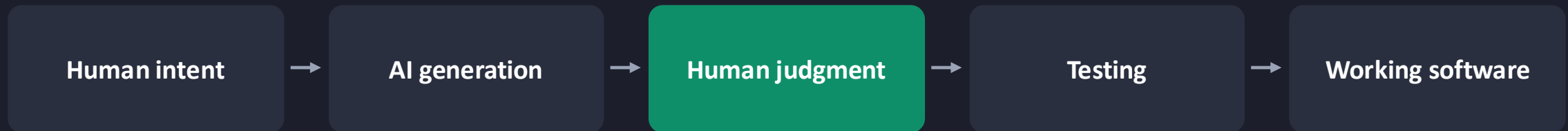
6 Was the improved solution actually better?

7 What did you learn that you wouldn't have by simply copying the code?



KEY TAKEAWAY

Vibe coding changes how we write software. It does not remove the need to understand it.



The most important skill isn't generating more code — it's knowing whether the code solves the right problem and produces the right result.

AI can generate code. You are responsible for determining whether it is correct.



RESOURCES

<https://docs.github.com/en/copilot/how-tos/set-up/install-copilot-extension>

<https://docs.github.com/en/copilot/get-started/about-github-copilot#get-access>

<https://github.com/copilot>

<https://docs.github.com/en/copilot/how-tos/copilot-on-github/set-up-copilot/enable-copilot/set-up-for-students>

<https://cloud.ibm.com/docs/watsonx-code-assistant>

